

# WebClient

## WebClient

Spring 5에서 도입된 리액티브/비동기 **HTTP** 클라이언트다.

Spring WebFlux 모듈에 포함되어 있으며, 높은 동시성이 필요한 상황에 적합하다.

**RestTemplate** : 요청을 보내고 응답이 올 때까지 스레드가 대기한다 → **Blocking**

**WebClient** : 응답을 기다리는 동안 스레드가 다른 일을 할 수 있다 → **Non-Blocking**

참고로 RestTemplate 은 Spring 5부터 유지보수 모드(maintenance mode)에 들어갔고, 동기 방식이 필요하다면 Spring 6.1에서 도입된 RestClient 가 권장된다.

## 왜 알아야 하는가

- 대규모 트래픽을 처리하는 서비스에서 주로 사용된다
- 비동기/리액티브 프로그래밍 개념을 이해하는 데 도움이 된다
- 동기 방식은 RestClient 로 충분하지만, 비동기·논블로킹 처리가 필요하다면 WebClient 를 선택하게 된다

## 전제 조건: 의존성 추가

RestTemplate 과 달리 별도의 의존성이 필요하다.

```
// build.gradle.kts
implementation("org.springframework.boot:spring-boot-starter-webflux")
```

이 프로젝트는 WebFlux를 사용하지 않으므로 해당 의존성이 없다.

## 1. 기본 설정

### Bean 등록

```
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient webClient() {
        return WebClient.builder()
```

```

        .baseUrl("https://api.example.com")
        .defaultHeader(HttpHeaders.CONTENT_TYPE,
MediaType.APPLICATION_JSON_VALUE)
        .build();
    }
}

```

## 타임아웃 설정

```

@Bean
public WebClient webClient() {
    HttpClient httpClient = HttpClient.create()
        .responseTimeout(Duration.ofSeconds(5))
        .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 3000);

    return WebClient.builder()
        .clientConnector(new ReactorClientHttpConnector(httpClient))
        .build();
}

```

WebClient 는 기본적으로 **Reactor Netty** 기반의 HttpClient 를 사용한다. 이처럼 ReactorClientHttpConnector 로 연결/응답 타임아웃을 세밀하게 제어할 수 있다.

## 2. GET 요청

### 비동기 (Mono / Flux)

```

// Mono: 단건 응답 (0 또는 1개)
Mono<NaverNewsResponse> responseMono = webClient.get()
    .uri("/v1/search/news.json?query={query}", "보건소")
    .header("X-Naver-Client-Id", clientId)
    .header("X-Naver-Client-Secret", clientSecret)
    .retrieve()
    .bodyToMono(NaverNewsResponse.class);

// 실제로 값을 받으려면 subscribe() 또는 block()
NaverNewsResponse response = responseMono.block(); // 동기로 대기

```

### Flux: 여러 건 응답

```

// Flux: 여러 건 응답 (0~N개)
Flux<Member> members = webClient.get()
    .uri("/members")
    .retrieve()

```

```
.bodyToFlux(Member.class);
```

```
List<Member> memberList = members.collectList().block();
```

subscribe() / block() 을 호출하기 전까지는 요청이 실제로 진행되지 않는다. Mono / Flux 는 실행 계획일 뿐이고, 구독하는 순간 비로소 동작한다. (이를 lazy 또는 cold publisher라고 한다.)

---

### 3. POST 요청

```
ResponseCreateMember response = webClient.post()
    .uri("/members")
    .contentType(MediaType.APPLICATION_JSON)
    .bodyValue(new RequestCreateMember("홍길동"))
    .retrieve()
    .bodyToMono(ResponseCreateMember.class)
    .block();
```

### 4. PUT / DELETE 요청

```
// PUT
webClient.put()
    .uri("/members/{id}", 1L)
    .bodyValue(updateRequest)
    .retrieve()
    .bodyToMono(Void.class)
    .block();

// DELETE
webClient.delete()
    .uri("/members/{id}", 1L)
    .retrieve()
    .bodyToMono(Void.class)
    .block();
```

### 5. 에러 처리

#### onStatus로 상태 코드별 처리

```
NaverNewsResponse response = webClient.get()
    .uri(url)
```

```

.retrieve()
.onStatus(HttpStatusCode::is4xxClientError, clientResponse -> {
    log.error("클라이언트 에러: {}", clientResponse.statusCode());
    return Mono.error(new RuntimeException("API 요청 실패"));
})
.onStatus(HttpStatusCode::is5xxServerError, clientResponse -> {
    log.error("서버 에러: {}", clientResponse.statusCode());
    return Mono.error(new RuntimeException("외부 서버 에러"));
})
.bodyToMono(NaverNewsResponse.class)
.block();

```

`retrieve()` 는 기본적으로 4xx/5xx 응답에 대해 `WebClientResponseException` 을 던진다. `onStatus()` 로 상태 코드별 처리를 직접 정의할 수 있으며, 더 세밀한 제어가 필요하다면 `exchangeToMono()` 를 사용할 수도 있다.

## 타임아웃 처리

```

webClient.get()
    .uri(url)
    .retrieve()
    .bodyToMono(String.class)
    .timeout(Duration.ofSeconds(5)) // 리액티브 레벨 타임아웃
    .onErrorReturn("타임아웃 발생 시 기본값")
    .block();

```

## 재시도 처리

```

webClient.get()
    .uri(url)
    .retrieve()
    .bodyToMono(String.class)
    .retryWhen(Retry.backoff(3, Duration.ofSeconds(1))) // 최대 3회, 백오프 재
시도
    .block();

```

리액티브 연산자(`retryWhen`, `timeout`, `onErrorResume` 등)를 체이닝해 재시도·폴백 로직을 선언적으로 구성할 수 있다는 점이 `WebClient` 의 강점이다.

## 6. 핵심 개념: Mono와 Flux

`Mono<T>` → 0 또는 1개의 결과 (단건 조회, 생성, 수정 응답)  
`Flux<T>` → 0 ~ N개의 결과 (목록 조회, 스트리밍)

```
.block()      → 동기로 결과 대기 (MVC에서 사용 시)
.subscribe()  → 비동기로 결과 처리 (WebFlux에서 사용 시)
.collectList() → Flux를 List로 변환
```

## block()을 쓰면 비동기의 의미가 없는 것 아닌가?

맞다. Spring MVC(이 프로젝트)에서 WebClient 를 쓰면서 .block() 을 호출하면 동기적으로 동작하므로 WebClient 의 비동기 이점을 살리지 못한다. 이런 경우 RestClient 가 더 적합하다.

특히 WebFlux 환경의 이벤트 루프 스레드( reactor-http-nio-\* ) 위에서 .block() 을 호출하면 예외가 발생하거나 스레드가 묶여 성능에 심각한 영향을 줄 수 있으므로 주의해야 한다.

## 7. 동기 vs 비동기 동작 방식

동기 (RestTemplate, RestClient):

요청 → [스레드 대기중...] → 응답 → 다음 작업

스레드 1:  요청      대기      응답

비동기 (WebClient):

요청 → 스레드 반환 → (다른 작업) → 응답 도착 → 콜백 실행

스레드 1:  요청      다른작업      응답콜백

## 8. 장단점

### 장점

- 비동기/논블로킹 → 스레드를 효율적으로 사용한다
- 높은 동시성 처리가 가능하다 (수천 개의 동시 요청)
- 스트리밍을 지원한다
- RestClient 와 비슷한 깔끔한 체이닝 API를 제공한다
- 리액티브 연산자로 강력한 조합이 가능하다 (retry, timeout, fallback)

### 단점

- 리액티브 프로그래밍 학습 곡선이 높다 (Mono, Flux, 연산자)
- 디버깅이 어렵다 (스택 트레이스가 비동기 경계에서 끊긴다)
- 테스트 작성이 상대적으로 복잡하다
- Spring MVC에서 .block() 으로만 쓰면 이점이 없다

- WebFlux 의존성이 추가로 필요하다

## 9. 언제 WebClient를 선택하는가

WebClient를 선택해야 하는 상황:

Spring WebFlux 기반 프로젝트  
한 번에 여러 외부 API를 동시 호출해야 할 때  
스트리밍 응답 (SSE, WebSocket)  
대량 트래픽에서 스레드 절약이 중요할 때

WebClient가 필요 없는 상황 (이 프로젝트):

Spring MVC 기반  
외부 API 호출이 단순하고 순차적  
리액티브 프로그래밍이 불필요  
→ RestClient로 충분

## 정리

| 클라이언트        | 방식                 | 도입                    | 권장 환경         |
|--------------|--------------------|-----------------------|---------------|
| RestTemplate | 동기 (Blocking)      | Spring 3 (현재 유지보수 모드) | 레거시           |
| RestClient   | 동기 (Blocking)      | Spring 6.1            | MVC, 단순 동기 호출 |
| WebClient    | 비동기 (Non-Blocking) | Spring 5              | WebFlux, 고동시성 |

WebClient의 핵심은 구독 전까지 실행되지 않는 **Mono / Flux** 와 **논블로킹 I/O** 다. 다만 Spring MVC에서 `.block()` 으로부터만 사용한다면 굳이 WebFlux 의존성을 추가할 이유가 없으니, 환경과 목적에 맞춰 RestClient 와 WebClient 중 선택하는 것이 좋다.